



Afterlife Maniac is an infinite runner arcade game in which you are stuck in your afterlife and want to go to heaven but first you need to cross the roads of hell and stay safe from the reckless drivers!

Afterlife Maniac is a stylized low-poly endless arcade game inspired by Crossy Road and Frogger. Players control a character trapped in the afterlife and must continuously move forward across procedurally generated roads filled with dangerous traffic. The objective is simple: survive as long as possible and achieve the highest score by advancing further into the level. As the game progresses, vehicle density increases and traffic moves faster, creating a more

challenging experience. The game features an isometric camera perspective, simple controls using the arrow keys, and a score system that rewards forward progression. The visual design uses a low-poly afterlife theme with custom-made player and vehicle models created in Blender, while atmospheric background music enhances the experience. The combination of procedural generation, escalating difficulty, and score chasing encourages replayability and competitive high-score attempts.

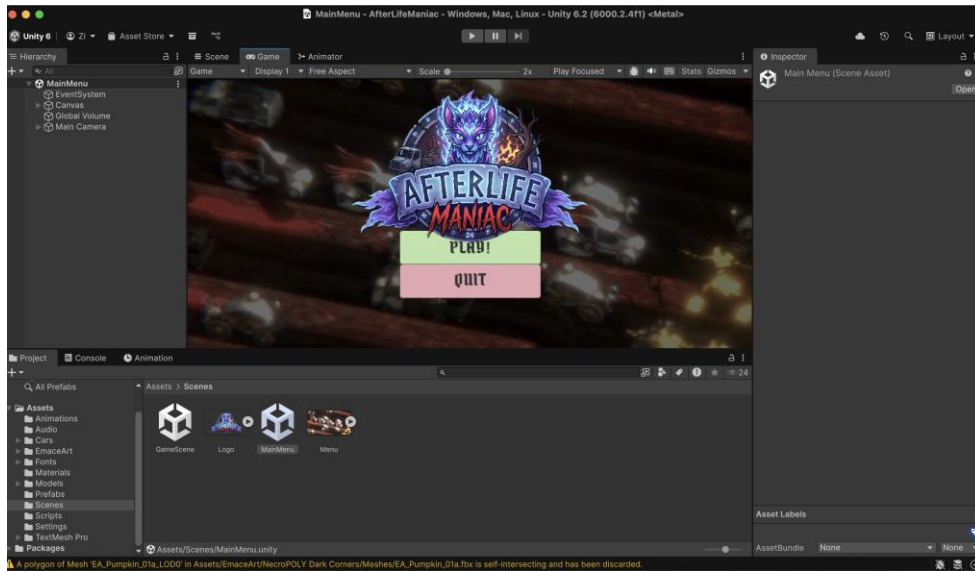
In order to download the project, first you can clone the repository–

<https://github.com/Klaf-ke/-School-MissileCmd>

Requirements:

1. Unity Hub
2. Unity Editor ver. 6000.2.4f1
3. Atleast 3 gb of free space on your disk

After cloning the code from github, open the project through unity hub.



Once opened the project, in project window go to Assets/Scenes and open it

Make sure scenes are loaded in build settings

Click on play to start the game!

Controls:

1. Arrow keys in the direction the player wants to move
2. R to restart

The game has a score which increases with each step forward by the player

High Score: High score gets stored and shown at the end of each session / after death

How does the game work?

GameManager.cs handles all the code including player controls, game loop and restarting the game

Spawning tiles:

In the game scene, there is an object called **GameManager** on which the main GameManager script is attached.

SpawnObstacle() function is used to spawn the vehicles and also they get destroyed once out of frame.

Road.cs handles the road spawning logic and the tress spawning

Prefabs:

In the prefabs folder, new obstacles and trees can be added easily by creating new prefabs of them and providing box collider, rigidbody followed by a vehicle tag.

Core Mechanics Framework

Physics

- Rigidbody-based player movement.
- Box colliders used for player and vehicle collision detection.
- Collision with vehicles results in immediate game over.
- Continuous forward movement influences scoring.

Internal Economy

- Score acts as the game's economy.
- Players earn points by progressing further into the level.
- No currency, upgrades, or purchasable items exist.

Progression

- No permanent upgrades.
- Difficulty progression occurs through:
 - Increased vehicle speed.
 - Increased traffic density.
 - Longer survival requirements.

Tactical Manoeuvring

- Players choose safe routes through traffic.
- Timing movement is essential.
- Risk versus reward decisions occur when crossing busy roads.
- Inspired by route-planning mechanics found in Crossy Road and Frogger.

Social Interaction

- No social or multiplayer systems.

- Competition occurs indirectly through high-score comparison.

Art Style

The game's art style is a stylized low-poly aesthetic designed to create a clear and readable gameplay experience while supporting the afterlife theme. The visual direction was inspired by Crossy Road's simple geometric forms and classic arcade crossing games such as Frogger. Custom vehicle models and the player character were created in Blender, allowing the visual style to remain consistent throughout the project.

Audio

The audio design focuses on creating atmosphere without distracting the player. Background music generated through ElevenLabs Music is used throughout gameplay to reinforce the supernatural afterlife setting. Audio was chosen to support the game's fast-paced arcade gameplay while maintaining immersion.

Prototype 1

- Basic player movement.
- Simple road crossing.
- Temporary placeholder objects.
- Testing focused on movement responsiveness.

Challenges Encountered

- Balancing the game loop.
- Ensuring difficulty increased at a fair rate.
- Maintaining player engagement over longer sessions.
- Fine-tuning procedural traffic generation.

Iterations

- Increased traffic density over time.
- Adjusted vehicle speed scaling.
- Improved procedural generation balance.
- Refined collision detection and scoring.

